21-07-2026
24261A6622

Experiment 1:

AIM: Write a python program to compute Central Tendency Measures: Mean,Median,Mode
Measure of Dispersion: Variance,Standard Deviation.

PROGRAM:

```
import numpy as np
import statistics as st
from math import isnan
from itertools import filterfalse
data=[9,8,5,4,3,7,6,6,7,8,8,8,4,4,2]
print("Dataset: ",data)
mean=st.mean(data)
mode=st.mode(data)
median=st.median(data)
variance=st.variance(data)
std_dev=st.stdev(data)
print("\nCentral Tendency Measures")
print("Mean: ",mean)
print("Mode: ",mode)
print("Median: ",median)
print("Variance: ",variance)
print("Standard Deviation: ",std_dev)
```

Output:

```
Dataset: [9, 8, 5, 4, 3, 7, 6, 6, 7, 8, 8, 8, 4, 4, 2]
Central Tendency Measures
Mean: 5.933333333333334
Mode: 8
Median: 6
Variance: 4.6380952380952385
Standard Deviation: 2.1536237457121516
```

Central Tendency Measures with Missing Data:

PROGRAM:

```
data1=[20.7,float('NaN'),19.2,18.3,float('NaN'),14,4]
sorted(data1)
```

```
print(data1)
print(st.median(data1))
sum(map(isnan,data1))
clean=list(filterfalse(isnan,data1))
print(clean)
print(sorted(clean))
print('median=',st.median(clean))
print('median_low=',st.median_low(clean))
print('median_high=',st.median_high(clean))
print('mode=',st.mode(clean))
print('mode=',st.mode([1,1,2,3,3,3,3,4]))
print('mode=',st.mode(["red","blue","blue","red","green","red","red"]))
print('multimode=',st.multimode('aabbbbbccddddeeffffgg'))
```

Output:

```
[20.7, nan, 19.2, 18.3, nan, 14, 4]
14
[20.7, 19.2, 18.3, 14, 4]
[4, 14, 18.3, 19.2, 20.7]
median= 18.3
median_low= 18.3
median_high= 18.3
mode= 20.7
mode= 3
mode= red
multimode= ['b', 'd']
```

Covariance and Correlation:

PROGRAM:

```
x=[10,15,7,8,8]
y=[40,35,50,50,29]
print('Covariance= ',np.cov(x,y))
print('Corelation= ',np.correlate(x,y))
```

Output:

```
Covariance= [[ 10.3 -12.85] [-12.85 85.7 ]]
Corelation= [1907]
```

28-07-2026
24261A6622

Experiment 2:

AIM: Study of Python Basic Libraries such as Statistics, Math, Numpy and Scipy

PROGRAM:

```
import numpy as np
arr=np.array([5,6,7])
print("Array: ",arr)
arr1=np.array([[2,9,3],[8,4,1]])
print("Array: ",arr1)
arr=np.array([[4,5,6,1.2],[9,10,4.7,8],[6,85,23,5],[4.6,3,5,3.6]])
print(arr)
sliced_arr=arr[:2,::2]
print(sliced_arr)
row_indices=[1,1,0,3]
column_indices=[3,2,1,0]
indexed_elements=arr[row_indices,column_indices]
print(indexed_elements)
a=np.array([[7,5],[6,9]])
b=np.array([[5,3],[2,3]])
print(a+1)
print(a)
print(a+b)
print(b-1)
print(a.sum())
s="Dhanunjaya"
print(s)
print(s[2])
print(s[-2])
print(s[1:5])
print(s[3:7:1])
```

Output:

```
Array:  [5 6 7]
Array:  [[2 9 3]
[8 4 1]]
[[ 4.   5.   6.   1.2]
[ 9.  10.   4.7  8. ]
```

```
[ 6.  85.  23.   5. ]
[ 4.6  3.   5.   3.6]]
[[4.  6. ]
[9.  4.7]]
[8.  4.7 5.  4.6]
[[ 8  6]
[ 7 10]]
[[7 5]
[6 9]]
[[12  8]
[ 8 12]]
[[4 2]
[1 2]]
27
Dhanunjaya
a
y
hanu
nunj
```

Basic Operations on Array:

PROGRAM:

```
x=np.array([[10,20,30,40],[5,10,15,20],[2,4,6,8]])
a=x.flatten()
print(a)
a[0]=99
print(a)
print(x)
x=np.array([[10,20,30,40],[5,10,15,20],[2,4,6,8]])
a1=x.ravel()
print(a1)
a1[0]=99
print(a1)
print(x)
x=np.array([10,20,30,40])
x.dtype
x=np.arange(5)
print(x)
```

```
y=np.arange(12).reshape(4,3)
print(y)
```

Output:

```
[10 20 30 40  5 10 15 20  2  4  6  8]
[99 20 30 40  5 10 15 20  2  4  6  8]
[[10 20 30 40]
[ 5 10 15 20]
[ 2  4  6  8]]
[10 20 30 40  5 10 15 20  2  4  6  8]
[99 20 30 40  5 10 15 20  2  4  6  8]
[[99 20 30 40]
[ 5 10 15 20]
[ 2  4  6  8]]
dtype('int32')
[0 1 2 3 4]
[[ 0  1  2]
[ 3  4  5]
[ 6  7  8]
[ 9 10 11]]
```

PROGRAM:

```
a=np.array([[3,4],[6,7]])
b=np.array([[11,2],[1,2]])
print(a@b)
print(a*b)
a=np.array([3,4,5,6,7])
b=np.arange(5)
print(a)
print(b)
print(a-b)
print(b**3)
print(a<6)
```

Output:

```
[[37 14]
[73 26]]
[[33  8]
```

```
[ 6 14]]
[3 4 5 6 7]
[0 1 2 3 4]
[3 3 3 3 3]
[ 0  1  8 27 64]
[ True  True  True False False]
```

Scipy Library:

PROGRAM:

```
from scipy import special as sp
a=sp.exp10(4)
b=sp.exp2(9)
c=sp.sindg(90)
print(a)
print(b)
print(c)
from scipy import linalg
x=np.array([[10,26],[5,10]])
print(x)
b=linalg.inv(x)
print(b)
```

Output:

```
10000.0
512.0
1.0
[[10 26]
[ 5 10]]
[[-0.33333333  0.86666667]
[ 0.16666667 -0.33333333]]
```

Differentiation using scipy library:

PROGRAM:

```
from scipy.differentiate import derivative
def my_function(x):
        return x**5+4*x+7
```

```
a=derivative(my_function,2)
print(a)
```

Output:

```
success: True
status: 0
df: 83.99999999999952
error: 1.9895196601282805e-13
nit: 2
nfev: 11
x: 2.0
```

Eigen Values and Eigen Vectors using scipy library:

PROGRAM:

```
arr=np.array([[2,7],[7,10]])
eg_val,eg_vect=linalg.eig(arr)
print(eg_val)
print(eg_vect)
```

Output:

```
[-2.06225775+0.j 14.06225775+0.j]
[[-0.86491009 -0.50192682]
[ 0.50192682 -0.86491009]]
```

Integration using scipy library:

PROGRAM:

```
from scipy import integrate
from math import sqrt
a=lambda x:x**4
b=integrate.quad(a,1,6)
print(b)
```

Output:

```
(1555.0, 1.7263968032921184e-11)
```

Experiment 3:

AIM: Study of Python Libraries for ML application such as Pandas and Matplotlib

Implementation of Pandas Library:

PROGRAM:

```
import pandas as pd
s=pd.Series(["Rohit","Virat","Dhoni","Bumrah"])
print(s)
print(len(s))
print(type(s))
s=pd.Series(["Rohit","Virat","Dhoni","Bumrah"],index=[3,4,1,2])
print(s)
print(len(s))
print(type(s))
fruit=["Mango","Kiwi","Apple","Cherry"]
cost=[200,300,400,100]
sales=pd.Series(cost,index=friut)
print(sales)
```

Output:

```
0     Rohit
1     Virat
2     Dhoni
3    Bumrah
dtype: object
4
<class 'pandas.core.series.Series'>
3     Rohit
4     Virat
1     Dhoni
2    Bumrah
dtype: object
4
<class 'pandas.core.series.Series'>
```

Data Frames using Pandas Library:

PROGRAM:

```
EmployeeData={'ID':[10,20,30,40],'Name':['Dhanunjaya','Suhail','Vignan','Yashwantt', 'Age':[20,20,20,19],
'City':[ 'Vijaywada', 'Anantapur', 'Adilabad', 'Hyderabad']}
EmpDB=pd.DataFrame(EmployeeData)
print(EmpDB)EmpDB=pd.DataFrame(EmployeeData,index=[1,2,3,4])
print(EmpDB)
```

Output:

```
   ID       Name          Age       City
0  10   Dhanunjaya    20     Vijaywada
1  20    Suhail           20     Anantapur
2  30    Vignan          20     Adilabad
3  40     Yashwantt    19     Hyderabad
   ID       Name          Age       City
1  10  Dhanunjaya    20     Vijaywada
2  20     Suhail         20      Anantapur
3  30     Vignan       20      Adilabad
4  40   Yashwantt     19      Hyderabad
```

Empty Data Frames using Pandas Library:

PROGRAM:

```
df=pd.DataFrame()
print(df)
print(type(df))
CD=['id','Emp','Sal','Exp']
df=pd.DataFrame(columns=CD)
print(df)
new_row=pd.DataFrame([{"id":101,"Emp":"Manager","Sal":100000,"Exp":3}])
df=pd.concat([df,new_row],ignore_index=True)
new_row=pd.DataFrame([{"id":102,"Emp":"SalesPerson","Sal":60000,"Exp":2}])
df=pd.concat([df,new_row],ignore_index=True)
new_row=pd.DataFrame([{"id":103,"Emp":"Clerk","Sal":10000,"Exp":3}])
df=pd.concat([df,new_row],ignore_index=True)
new_row=pd.DataFrame([{"id":104,"Emp":"President","Sal":1000000,"Exp":10}])
df=pd.concat([df,new_row],ignore_index=True)
print(df)
```

Output:

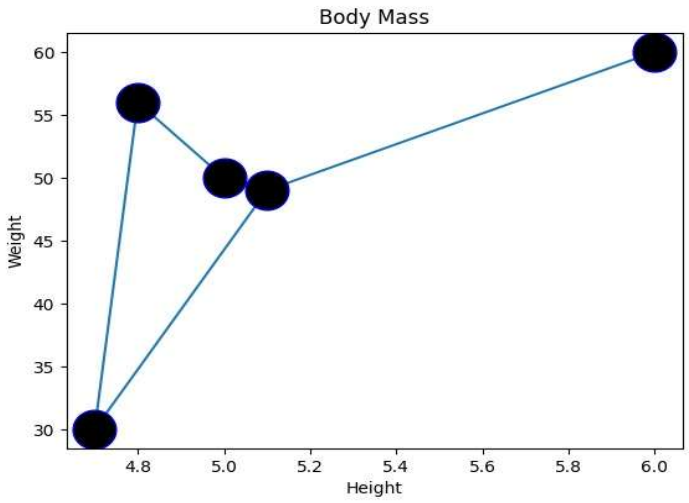
```
Empty DataFrame
Columns: []
Index: []
<class 'pandas.core.frame.DataFrame'>
Empty DataFrame
Columns: [id, Emp, Sal, Exp]
Index: []
    id         Emp      Sal  Exp
0  101     Manager   100000    3
1  102  SalesPerson    60000    2
2  103       Clerk    10000    3
3  104   President  1000000   10
```

Implementation of Matplotlib Library:

PROGRAM:

```
import matplotlib.pyplot as plt
import numpy as np
x=np.array([5,4.8,4.7,5.1,6])
y=np.array([50,56,30,49,60])
plt.plot(x,y,marker='o',ms=25,mec='b',mfc='black')
plt.xlabel("Height")
plt.ylabel("Weight")
plt.title("Body Mass")
plt.show()
```

Output:

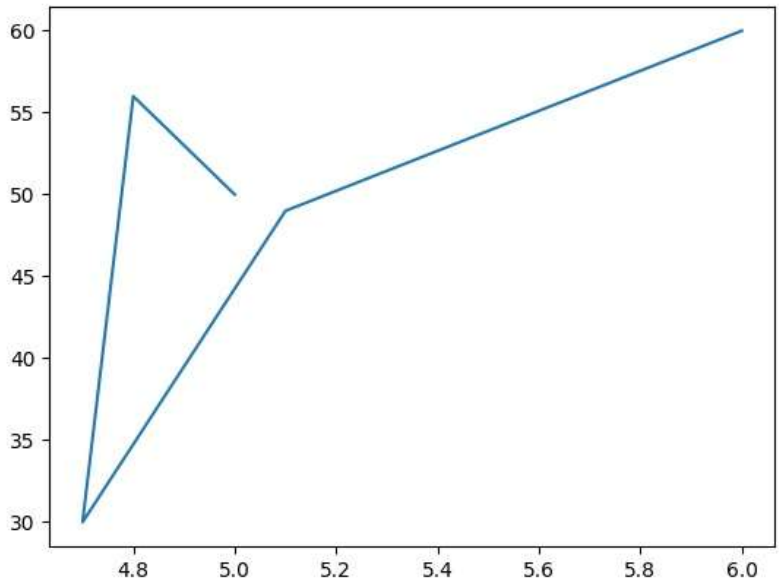


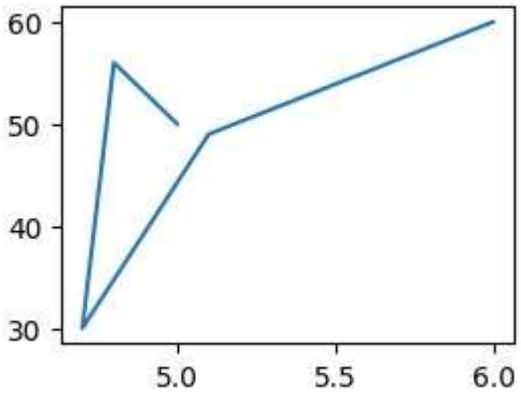
Subplot using Matplotlib Library:

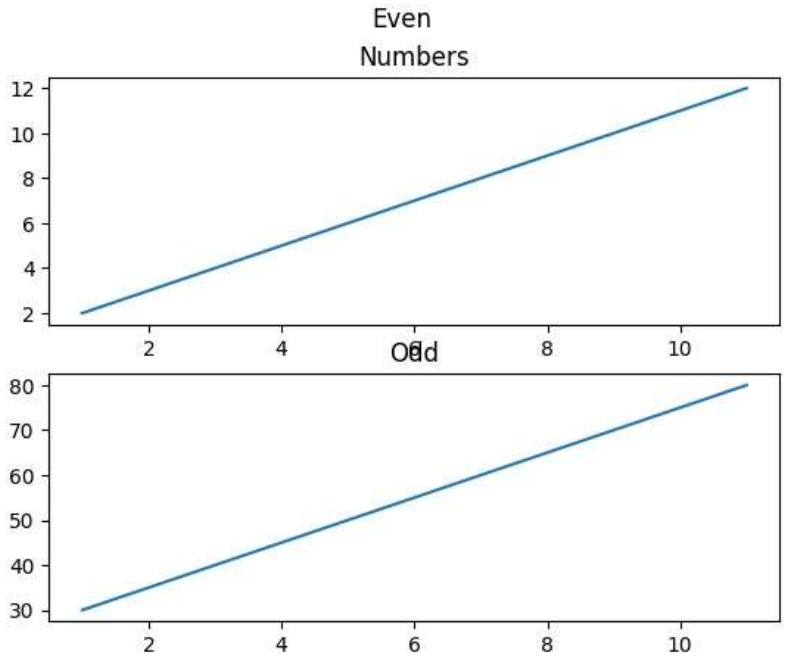
PROGRAM:

```
plt.subplot(1,1,1)
plt.plot(x,y)
plt.show()
plt.subplot(2,2,2)
plt.plot(x,y)
plt.show()
x=np.array([1,3,5,7,9,11])
y=np.array([2,4,6,8,10,12])
plt.subplot(2,1,1)
plt.plot(x,y)
plt.title("Numbers")
x1=np.array([1,3,5,7,9,11])
y1=np.array([30,40,50,60,70,80])
plt.subplot(2,1,2) plt.plot(x1,y1)
plt.title("Odd")
plt.suptitle("Even")
plt.show()
```

Output:





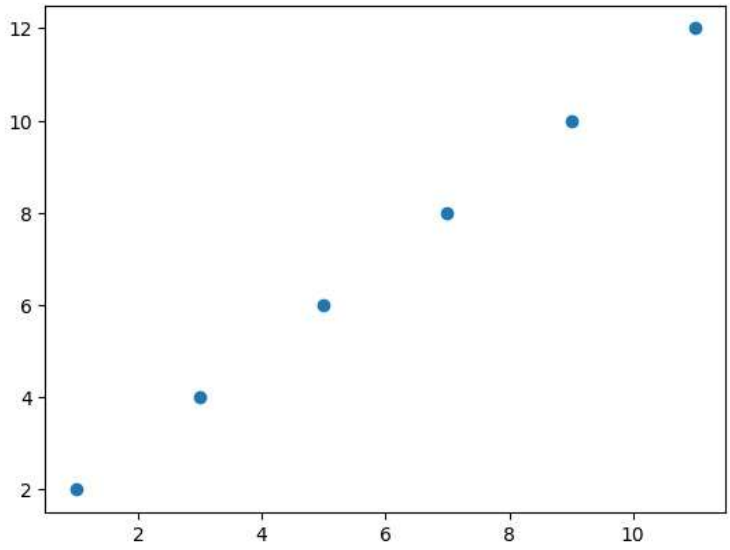


Scatter Plot using using Matplotlib Library:

PROGRAM:

```
x=np.array([1,3,5,7,9,11])
y=np.array([2,4,6,8,10,12])
plt.scatter(x,y)
plt.show()
```

Output:

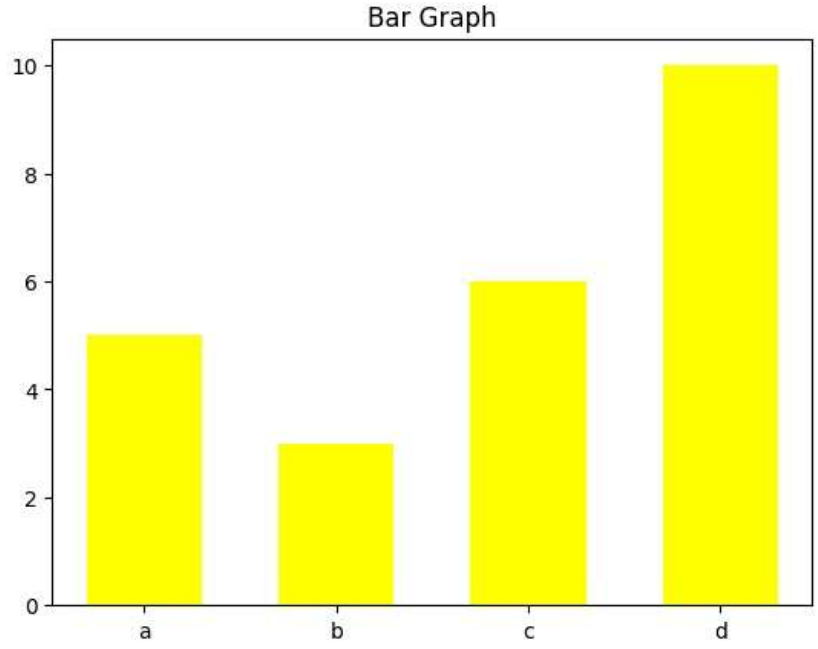


BarGraph using Matplotlib Library:

PROGRAM:

```
x=np.array(["a","b","c","d"])
y=np.array([5,3,6,10])
plt.bar(x,y,color='yellow',width=0.6)
plt.title("Bar Graph")
plt.show()
```

Output:

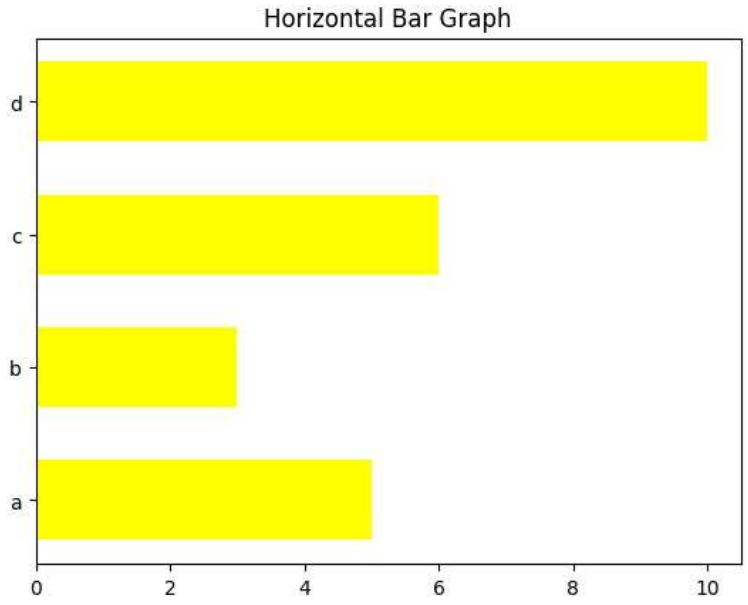


Horizontal BarGraph using Matplotlib Library:

PROGRAM:

```
x=np.array(["a","b","c","d"])
y=np.array([5,3,6,10])
plt.barh(x,y,color='yellow',height=0.6)
plt.title("Horizontal Bar Graph")
plt.show()
```

Output:

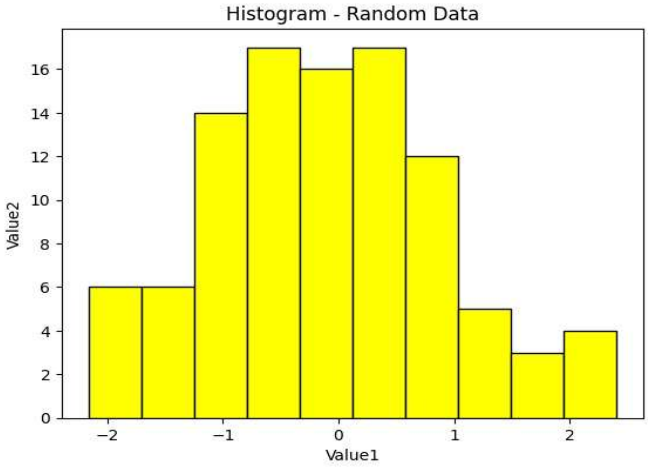


Histogram using Matplotlib Library:

PROGRAM:

```
data=np.random.randn(100)
plt.hist(data,bins=10,color='yellow',edgecolor='black',label="Random")
plt.title("Histogram - Random Data")
plt.xlabel("Value1")
plt.ylabel("Value2")
plt.show()
```

Output:

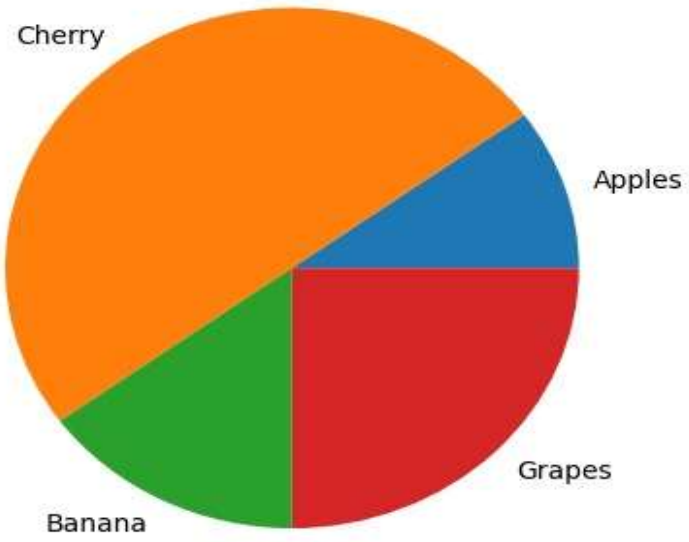


Pie Chart using Matplotlib Library:

PROGRAM:

```
y=np.array([10,50,15,25])
myl=['Apples','Cherry','Banana','Grapes']
plt.pie(y,labels=myl)
plt.show()
```

Output:



Experiment 4:

AIM: Write a python program to implement Simple Linear Regression for House Price Prediction

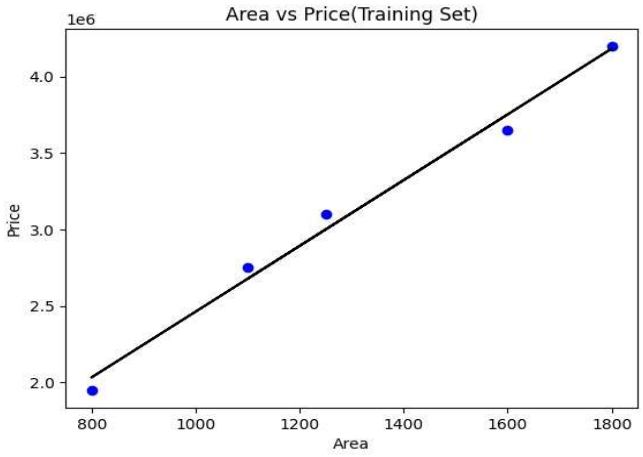
PROGRAM:

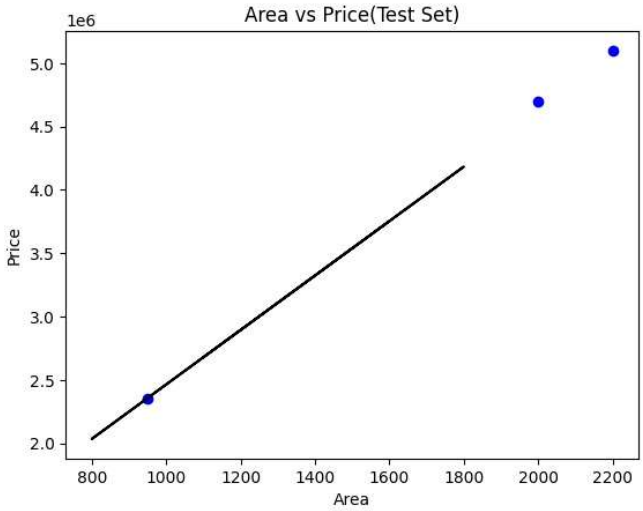
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn import linear_model data=pd.read_csv("C:/Users/MPIT/Downloads/house_price.csv”)
data
x=data.iloc[:, :-1].values
x
y=data.iloc[:,1].values
y
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(x,y,test_size=0.3,random_state=0)
X_train
X_test
from sklearn.linear_model import LinearRegression
regressor=LinearRegression()
regressor.fit(X_train,y_train)
y_pred=regressor.predict(X_test)
y_pred
y_test
plt.scatter(X_train,y_train,color='blue')
plt.plot(X_train,regressor.predict(X_train),color='black')
plt.title("Area vs Price(Training Set)")
plt.xlabel("Area")
plt.ylabel("Price")
plt.show()
plt.scatter(X_test,y_test,color='blue')
plt.plot(X_train,regressor.predict(X_train),color='black')
plt.title("Area vs Price(Test Set)")
plt.xlabel("Area")
plt.ylabel("Price")
plt.show()
print("Coeffecient:",regressor.coef_)
print("Intercept:",regressor.intercept_)
```

```
p=regressor.predict(np.array([[4500]]))
print("Predicted Price=",p[0])
from sklearn.metrics import mean_absolute_error,mean_squared_error
mae=mean_absolute_error(y_test,y_pred)
print(“Mean Absolute Error: “,mae)
mse=mean_squared_error(y_test,y_pred)
print(“Mean Squared Error: “,mse)
rmse=np.sqrt(mse)
print(“Root Mean Squared Error: “,rmse)
```

Output:

```
   Area_sqft  Price
0    1800      4200000
1    950       2350000
2   2200       5100000
3   1250       3100000
4   1600       3650000
5   800        1950000
6   2000       4700000
7   1100       2750000
array([[1800],
           [ 950],
           [2200],
           [1250],
           [1600],
           [ 800],
           [2000],
           [1100]], dtype=int64)
array([4200000, 2350000, 5100000, 3100000, 3650000, 1950000, 4700000, 2750000], dtype=int64)
array([[1100],
           [1250],
           [1800],
           [ 800],
           [1600]], dtype=int64)
array([[2000],
           [2200],
           [ 950]], dtype=int64)
array([4613172.46835443, 5043077.53164557, 2356170.88607595])
array([4700000, 5100000, 2350000], dtype=int64)
```





Coeffecient: [2149.52531646]

Intercept: 314121.8354430385

Predicted Price= 9986985.75949367

Mean Absolute Error: 49973.62869198341

Mean Squared Error: 3605755830.061998

Root Mean Squared Error: 60047.94609361754

Experiment 5:

AIM: Implementation of Multiple Linear Regression for House Price Prediction using sklearn

PROGRAM:

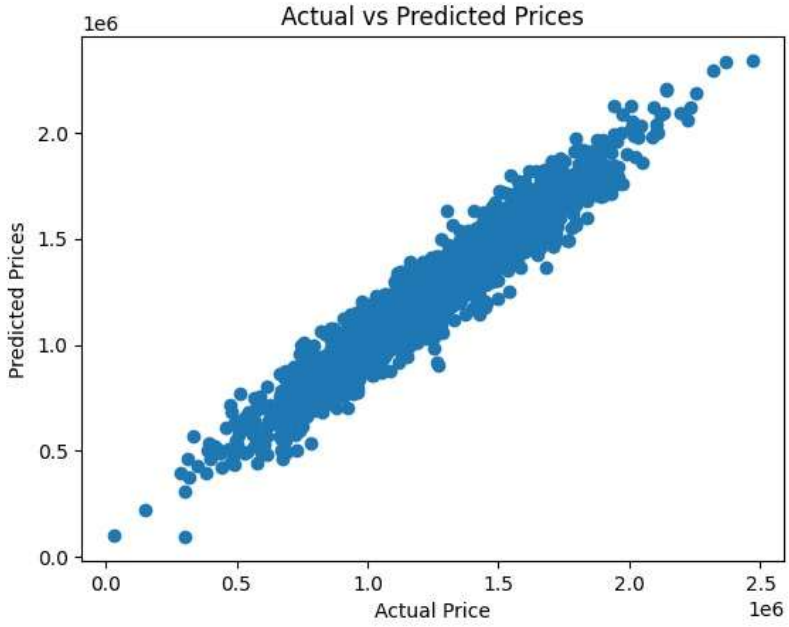
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn import metrics
USAhousing=pd.read_csv("C:/Users/MPIT/Downloads/USA_Housing.csv")
print(USAhousing.head())
X=USAhousing[['Avg. Area Income','Avg. Area House Age','Avg. Area Number of Rooms,
              'Avg. Area Number of Bedrooms','Area Population']]
y=USAhousing['Price']
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.3,random_state=0)
lm=LinearRegression()
lm.fit(X_train,y_train)
predictions=lm.predict(X_test)
plt.scatter(y_test,predictions)
plt.xlabel('Actual Price')
plt.ylabel('Predicted Prices')
plt.title('Actual vs Predicted Prices')
plt.show()
print('MAE(Mean Absolute Error):',metrics.mean_absolute_error(y_test,predictions))
print('MSE(Mean Squared Error):',metrics.mean_squared_error(y_test,predictions))
print('RMSE(Root Mean Squared Error):',np.sqrt(metrics.mean_squared_error(y_test,predictions))
```

Output:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms
0	79545.458574	5.682861	7.009188
1	79248.642455	6.002900	6.730821
2	61287.067179	5.865890	8.512727
3	63345.240046	7.188236	5.586729
4	59982.197226	5.040555	7.839388

	Avg. Area Number of Bedrooms	Area Population	Price
0	4.09	23086.800503	1.059034e+06
1	3.09	40173.072174	1.505891e+06
2	5.13	36882.159400	1.058988e+06
3	3.26	34310.242831	1.260617e+06
4	4.23	26354.109472	6.309435e+05

	Address
0	208 chael Ferry Apt. 674\nLaurabury, NE 3701...
1	188 Johnson Views Suite 079\nLake Kathleen, CA...
2	9127 Elizabeth Stravenue\nDanieltown, WI 06482...
3	USS Barnett\nFPO AP 44820
4	USNS Raymond\nFPO AE 09386



MAE(Mean Absolute Error): 81563.14733992997
MSE(Mean Squared Error): 10337337828.264475
RMSE(Root Mean Absolute Error): 101672.6995228536

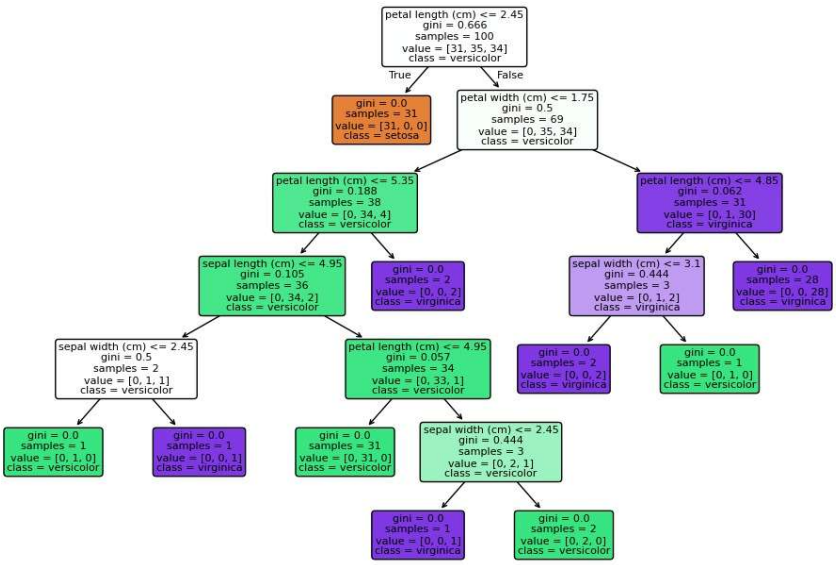
Experiment 6:

A) AIM: Implementation of Decision tree using sklearn

PROGRAM:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier,plot_tree
from sklearn.metrics import classification_report,accuracy_score,confusion_matrix
import matplotlib.pyplot as plt
iris=load_iris()
X=iris.data
y=iris.target
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.33,random_state=42)
treemodel=DecisionTreeClassifier(random_state=42)
treemodel.fit(X_train,y_train)
y_pred=treemodel.predict(X_test)
plt.figure(figsize=(12,8))
plot_tree(treemodel,filled=True,feature_names=iris.feature_names,
        class_names=iris.target_names,rounded=True)
plt.show()
print("Classification Report:")
print(classification_report(y_test,y_pred,target_names=iris.target_names))
print("Confusion Matrix:")
print(confusion_matrix(y_test,y_pred))
print("Accuracy Score:") print(accuracy_score(y_test,y_pred))
```

Output:



Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	0.94	1.00	0.97	15
virginica	1.00	0.94	0.97	16
accuracy			0.98	50
macro avg	0.98	0.98	0.98	50
weighted avg	0.98	0.98	0.98	50

Confusion Matrix: [[19 0 0]
[0 15 0]
[0 1 15]]

Accuracy Score:
0.98

Experiment 6:

B) AIM: Implementation of Decision tree using sklearn and its parameter tuning

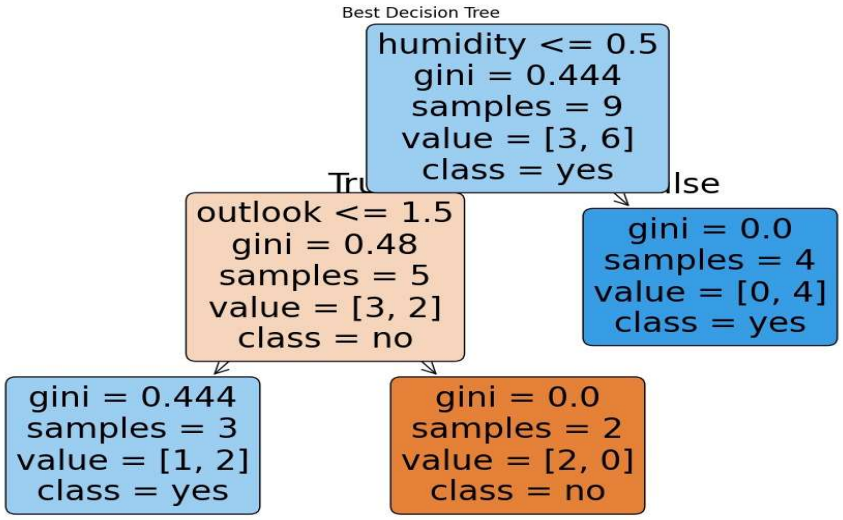
PROGRAM:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import classification_report, accuracy_score, confusion_matrix
import matplotlib.pyplot as plt
df = pd.read_csv("C:/Users/MPIT/Downloads/PlayTennis.csv")
label_encoders = {}
for column in df.columns:
        le = LabelEncoder()
        df[column] = le.fit_transform(df[column])
        label_encoders[column] = le
X = df.drop('play', axis=1)
y = df['play']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.33, random_state=42)
hyperparameters = [
        {'criterion': 'gini', 'max_depth': None, 'min_samples_split': 2, 'min_samples_leaf':1},
        {'criterion': 'entropy', 'max_depth': 4, 'min_samples_split': 2, 'min_sample s_leaf':1},
        {'criterion': 'gini', 'max_depth': 6, 'min_samples_split': 5, 'min_samples_leaf':2},
        {'criterion': 'entropy', 'max_depth': 8, 'min_samples_split': 10, 'min_samples_leaf':4},
]
best_accuracy = 0
best_params = None
best_tree = None
for params in hyperparameters:
        tree = DecisionTreeClassifier(**params, random_state=42)
        tree.fit(X_train, y_train)
        y_pred = tree.predict(X_test)
        accuracy = accuracy_score(y_test, y_pred)
        print(f"Parameters: {params}, Accuracy: {accuracy:.4f}")
        if accuracy > best_accuracy:
                best_accuracy = accuracy
                best_params = params
                best_tree = tree
print(f"\nBest Parameters: {best_params}, Best Accuracy: {best_accuracy:.4f}")
plt.figure(figsize=(12, 8))
plot_tree(best_tree, filled=True, feature_names=list(X.columns),
class_names=list(label_encoders['play'].classes_), rounded=True)
plt.title('Best Decision Tree')
```

```
plt.show()
y_pred_best = best_tree.predict(X_test)
print("Best Decision Tree - Classification Report:")
print(classification_report(y_test, y_pred_best, target_names=label_encoders['play'].classes_))
print("Best Decision Tree - Confusion Matrix:")
print(confusion_matrix(y_test, y_pred_best))
print("Best Decision Tree - Accuracy Score:")
print(accuracy_score(y_test, y_pred_best))
```

Output:

```
Parameters: {'criterion': 'gini', 'max_depth': None, 'min_samples_split': 2, 'min _samples_leaf': 1},
        Accuracy: 0.6000
Parameters: {'criterion': 'entropy', 'max_depth': 4, 'min_samples_split': 2, 'min _samples_leaf': 1},
        Accuracy: 0.6000
Parameters: {'criterion': 'gini', 'max_depth': 6, 'min_samples_split': 5, 'min_sa mples_leaf': 2},
        Accuracy: 0.8000
Parameters: {'criterion': 'entropy', 'max_depth': 8, 'min_samples_split': 10, 'mi n_samples_leaf': 4},
        Accuracy: 0.6000
Best Parameters: {'criterion': 'gini', 'max_depth': 6, 'min_samples_split': 5, 'm in_samples_leaf': 2},
        Best Accuracy: 0.8000
```




```
Best Decision Tree - Classification Report:
                    precision    recall  f1-score   support

        no              1.00      0.50      0.67         2
       yes              0.75      1.00      0.86         3

    accuracy                           0.80         5
   macro avg       0.88      0.75      0.76         5
weighted avg       0.85      0.80      0.78         5

Best Decision Tree - Confusion Matrix:
[[1 1]
 [0 3]]
Best Decision Tree - Accuracy Score:
0.8
```

Experiment 7:

AIM: Implementation of KNN using sklearn

PROGRAM:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
iris = load_iris()
data = iris.data
target = iris.target
X_train, X_test, y_train, y_test = train_test_split(data, target, test_size=0.3)
clf = KNeighborsClassifier(n_neighbors=5)
clf.fit(X_train, y_train)
predict = clf.predict(X_test)
print(f"Scikit-learn KNN classifier accuracy: {accuracy_score(y_test, predict)}")
print(classification_report(y_test, predict))
```

Output:

```
Scikit-learn KNN classifier accuracy: 0.9555555555555556
              precision    recall  f1-score   support
0                 1.00      1.00      1.00        14
1                 0.92      0.92      0.92        13
2                 0.94      0.94      0.94        18
accuracy                              0.96        45
macro avg         0.96      0.96      0.96        45
weighted avg      0.96      0.96      0.96        45
```

Experiment 8:

AIM: Implementation of Logistic Regression using sklearn

PROGRAM:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target
df['target_name'] = df['target'].apply(lambda x: iris.target_names[x])
print("First five rows of the dataset:\n")
print(df.head())
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
model = LogisticRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=iris.target_names))
sample = [[5.9, 3.0, 5.1, 1.8]]
predicted_class = model.predict(sample)
print("pc=",predicted_class)
print("\nPredicted Flower Type:", iris.target_names[int(predicted_class)])
```

Output:

First five rows of the dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	target	target_name
0	0	setosa
1	0	setosa
2	0	setosa
3	0	setosa
4	0	setosa

Accuracy: 1.0
Confusion Matrix:
[[19 0 0]
[0 13 0]
[0 0 13]]
Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	1.00	1.00	13
virginica	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

pc= [2]
Predicted Flower Type: virginica

Experiment 9:

AIM: Implementation of SVM using sklearn

PROGRAM:

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.svmimport SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target
print("First five rows of the dataset:\n")
print(df.head())
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
model = SVC(kernel='linear')
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print("\nAccuracy:", accuracy_score(y_test, y_pred))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=iris.target_names))
```

Output:

First five rows of the dataset:

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	\
0	5.1	3.5	1.4	0.2	
1	4.9	3.0	1.4	0.2	
2	4.7	3.2	1.3	0.2	
3	4.6	3.1	1.5	0.2	
4	5.0	3.6	1.4	0.2	

	target
0	0
1	0
2	0
3	0
4	0

Accuracy: 1.0

Confusion Matrix:

```
[[19  0  0]
[ 0 13  0]
[ 0  0 13]]
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	19
versicolor	1.00	1.00	1.00	13
virginica	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Experiment 10:

AIM: Implementation of K-Means Clustering

PROGRAM:

```
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt
import seaborn as sns
data = pd.read_csv("E:\users\Desktop\Iris.csv")
X = data.drop(columns=['Species'])
y = data['Species']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(X_scaled)
labels = kmeans.labels_
labels
label_map = {}
for cluster in range(3):
    species_in_cluster = y[labels == cluster]
    most_common_species = species_in_cluster.mode()[0]
    label_map[cluster] = most_common_species
label_map
predicted_species = [label_map[label] for label in labels]
correct_labels = sum(y == predicted_species)
print("Result: %d out of %d samples were correctly labeled." % (correct_labels, y.size))
print("Accuracy score: {0:0.2f}".format(correct_labels / float(y.size)))
```

Output:

```
Result: 143 out of 150 samples were correctly labeled.
Accuracy score: 0.95
```